FRUTON User Manual

Framework for Reconstruction, UniProt alignment, and Topology-Oriented protein Normalization

Practical guide to inputs, execution, decision-making, workbook interpretation, outputs, and troubleshooting

Repository context: stack_protein_prep

Prepared as a user-facing manual rather than an internal note.

1. What FRUTON is for

FRUTON is a state-driven protein-preparation framework for turning a list of protein structures into inspectable, downstream-ready outputs. It does not only clean structures and continue. It also records what was found, what decisions were made, which outputs were retained, and where every important intermediate file was written.

This manual is written as a usage document rather than an internal development note. The goal is to answer practical questions: what input FRUTON expects, how it is run, what each stage contributes, how the variant logic works, how the workbook should be read, and where to look when a record fails or looks suspicious.

A short explanation of the name and logo is still useful because FRUTON is a named framework rather than an anonymous script. That context belongs in the manual, but only as background. The main focus remains operation and interpretation.

2. Name, logo, and namesake

The FRUTON name is an acronym for Framework for Reconstruction, UniProt alignment, and Topology-Oriented protein Normalization. The logo uses a stylized strawberry shaped as the letter F. The outer form makes the mark immediately memorable, while the internal seed motifs evoke simplified protein secondary-structure elements rather than generic decoration.

The explanation should stay short. For a user, the important point is that the logo is readable before it is explained: fruit-like shape, F-shaped contour, and protein-themed interior. The explanation simply adds a second layer rather than rescuing a weak symbol.

The name also refers to Joseph S. Fruton, a Polish-born biochemist associated with protein chemistry and the history of biochemistry. That gives the framework a scientific namesake and makes the title feel like a real instrument rather than a disposable folder of scripts.

Practical takeaway

The logo and name are part of the project identity, but they do not replace technical documentation.

In user-facing contexts, keep the identity explanation brief and move quickly to workflow, outputs, and interpretation.

In README or presentation contexts, the logo can serve as a signet; in the manual it should remain background context.

3. Minimal required input

At minimum, FRUTON expects a protein data directory containing a CSV file with a pdb_id column and, optionally, a range column. Each row defines one PDB entry to be synchronized and processed. A range can be used to keep only a selected residue interval for the polymer ATOM records while preserving water and other selected records according to the module rules.

The practical starting point is therefore not a single structure file but an input table. FRUTON is designed to operate over a small or medium batch of proteins in one run, while still keeping the state of each protein separate and inspectable.

Required item	Meaning
pdb_ids.csv	Input table with at least a pdb_id column; range is optional.
data/proteins/	Central working directory for downloaded structures, module outputs, logs, JSON state, and Excel workbook.
Python environment	Environment with required scientific and utility dependencies for FRUTON modules.
External tools	Depending on stage: MAFFT, pdb2pqr, MODELLER, and other configured tools.

Example CSV

```
pdb_id,range
1ABC,10-280
2XYZ,
3DEF,5-190
```

4. Expected working structure

FRUTON writes a structured directory tree under data/proteins. Each PDB ID gets its own working directory, and central reports such as pipeline.json and pipeline.xlsx are written at the shared data/proteins level. The user should think of the project as a stateful workspace rather than a one-shot command.

The exact tree grows as the run proceeds, but a typical successful record accumulates raw structure files, FASTA files, alignment files, component splits, prepared outputs, and per-module logs.

Typical layout

```
data/proteins/
  pdb_ids.csv
  pipeline.json
  pipeline.xlsx
  logs/
    <PDB_ID>/
      filler.log
      protonation.log
      ...
  <PDB_ID>/
    <PDB_ID>.pdb
    fasta/
      PDB-<PDB_ID>-SEQRES.fasta
      PDB-<PDB_ID>-ATOM.fasta
      UniProt_<UNIPROT_ID>.fasta
      alignments/
        *.input.fasta
        *.aln.fasta
        *.mapping.tsv
        *.png
    components/
      <PDB_ID>_protein.pdb
      <PDB_ID>_water.pdb
      <PDB_ID>_ligand.pdb
      <PDB_ID>_metal.pdb
    prepared/
      single/
      gaps/
      best_complete/ or large_gap_complete/
```

5. How to run FRUTON

FRUTON is orchestrated from the runner script scripts/fruton.py. In practical terms, the user prepares the input CSV and working directory, activates the project environment, and launches the runner. The runner then executes the defined module sequence, updates the flat pipeline state, writes logs, and exports the workbook.

The exact command can differ depending on how the repository is installed, but the operational pattern is stable: run the FRUTON runner from the project context so that data/proteins can be found and updated in place.

Operational checklist

Confirm that pdb_ids.csv exists and contains the intended PDB IDs and ranges.

Confirm that required external tools are available in the active environment.

Run the FRUTON runner from the repository context.

After completion, inspect the terminal summary, pipeline.xlsx, pipeline.json, and logs/ directory.

6. What happens during the run

The runner follows a fixed high-level order. First it synchronizes structures from the CSV list. Then it creates FASTA context, aligns local chains against UniProt, handles insertion codes, splits structure components, evaluates gaps, decides whether filling is needed, runs the chemistry route for retained variants, and finally writes JSON and XLSX summaries.

The key point is that FRUTON is not merely a linear transformer. It is a linear runner with branching decisions inside it. That means two records in the same batch may share the same early steps but diverge later because their structural evidence differs.

Stage	User interpretation
PDB synchronization	Downloads missing PDB files and preserves requested versus observed residue bounds.
FASTA generation	Creates SEQRES- and ATOM-derived FASTA files and obtains UniProt context.
Sequence alignment	Runs chain-specific MAFFT alignments and produces mapping files and optional images.
Insertion-code handling	Normalizes insertion-code cases before downstream structural work.
Component splitting	Separates protein, water, ligand, metal, and artifact outputs.
Gap detection	Uses numbering discontinuities together with peptide C-N connectivity checks.
Filler logic	Chooses skip, MODELLER, or AlphaFold-oriented completion route.
Chemistry route	Runs protonation, AMBER renaming, termini conversion, and prepared-structure assembly.
Reporting	Updates pipeline.json and pipeline.xlsx and prints the FRUTON summary.

7. FRUTON as a decision-making framework

The most important idea for a user is that FRUTON makes explicit decisions rather than silently hiding them inside file names or control flow. The flat state model records both process status and interpretation fields such as gap sizes, retained-variant policy, final prepared variant, and the summary of available model families.

This matters because a user often does not only want a final file. They also want to know whether that file came from the original structure, a gap-preserving branch, a MODELLER completion, or a large-gap completion route. FRUTON keeps that information visible in the workbook and JSON state.

Field	Meaning for the user
n_gaps / gap_sizes / has_gaps	How much structural incompleteness was detected.
variant.policy	Which retention logic FRUTON applied to the record.
variant.retain_gaps	Whether an explicit gaps route remains part of the final result.
variant.retain_modeller	Whether a MODELLER-based complete route was retained.
variant.retain_alphafold	Whether an AlphaFold-based complete route was retained.
prepared_structure.variant	Which final variant label the user should expect to see.
available_models	Compact summary of structural families that were actually produced.

8. Understanding the variant policy

FRUTON does not assume that one final structure is always enough. Instead it distinguishes structural families and records which ones were retained. This policy is what makes the workbook useful: the user can understand not only that a record succeeded, but what kind of success it was.

The basic logic is compact. If no relevant gaps are present, the record can remain a single best-available route. If gaps are present, an explicit gaps route is retained. If a complete model is also available, FRUTON distinguishes whether it corresponds to a smaller-gap best_complete case or a larger-gap large_gap_complete case.

Situation	Policy / variant	Meaning
No gaps	single	Use the best available initial structure route.
Gaps present, no complete model	gaps_only_no_complete_model	Keep the explicit gaps route only.
Gaps present, complete model available, smaller gap case	gaps_plus_best_complete	Retain the gaps route and a complete route suited to moderate missing regions.
Gaps present, complete model available, large-gap case	gaps_plus_large_gap_complete	Retain the gaps route and a more cautious complete route associated with larger missing regions.

9. Why the gaps route is separate

For a user, the gaps route should be understood as a chemically safer branch for disconnected or structurally incomplete systems. Rather than pushing one disconnected multi-fragment structure directly into protonation, FRUTON can split the raw protein into physically connected backbone fragments, protonate the fragments separately, continue chemistry on those fragment outputs, and only then merge or assemble the retained result.

This matters because protonation and downstream chemistry can become misleading when artificial discontinuities are treated as if they were one intact polymer. The separate gaps route preserves interpretability and makes the structural compromise visible instead of burying it.

10. How to use pipeline.xlsx

pipeline.xlsx is one of the main products of a FRUTON run. It is not an optional afterthought. JSON is useful for machine persistence, but the workbook is the main human review surface. A user should expect to spend time in the workbook after each meaningful run.

The workbook exists to answer three practical questions quickly: what is in the system, what happened during preparation, and which structural families are available in the end. The overview sheet is therefore the first place to start.

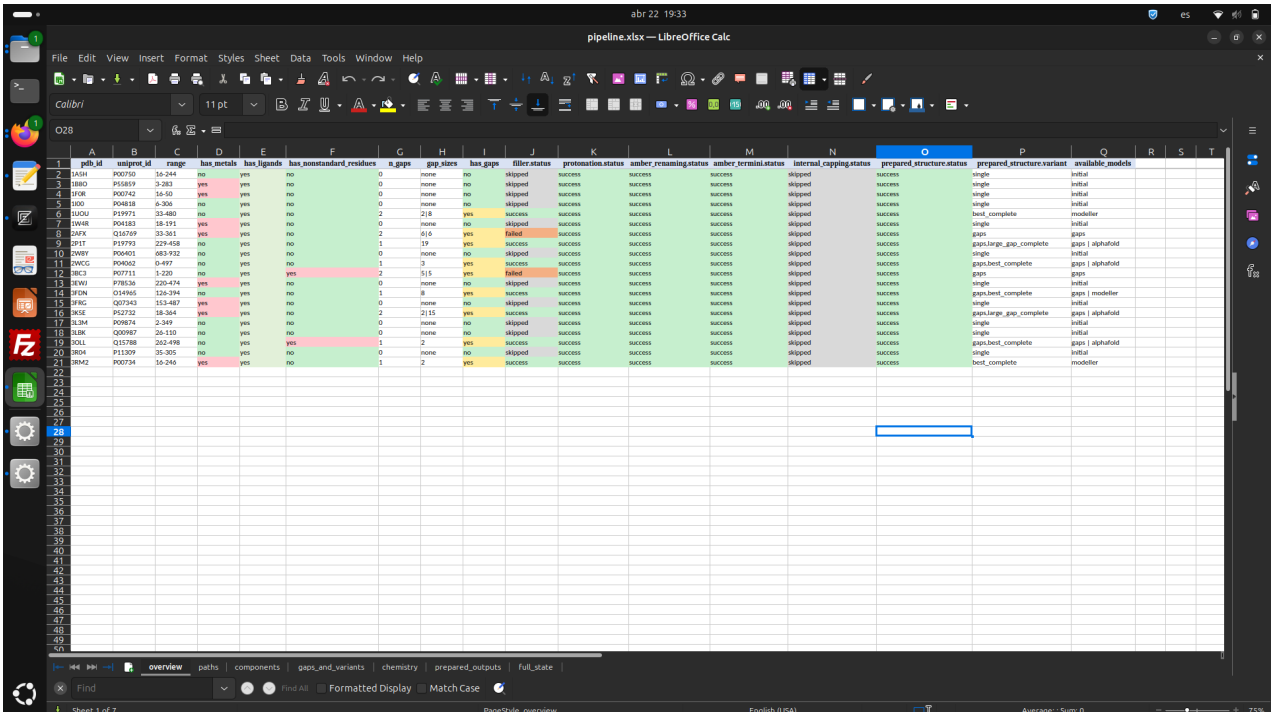
How to read the overview sheet

Start from the identifier columns: pdb_id, uniprot_id, and range. Then read the composition flags: has_metals, has_ligands, has_nonstandard_residues. After that, inspect the gap and status region:

n_gaps, gap_sizes, has_gaps, filler.status, protonation.status, AMBER-renaming status, termini status, internal-capping status, and prepared-structure status.

Finally, inspect prepared_structure.variant and available_models. Those two columns summarize the final structural outcome. They tell the user which route survived and which structural families are actually available on disk.

Workbook example



pdb_id	uniprot_id	range	has_metals	has_ligands	has_nonstandard_residues	n_gaps	gap_sizes	has_gaps	filler.status	protonation.status	amber_renaming.status	amber_termini.status	internal_capping.status	prepared_structure.status	prepared_structure.variant	available_models
1A3H	P00750	16-244	no	yes	no	0	none	no	skipped	success	success	success	skipped	success	single	initial
1BBO	P55859	3-283	yes	yes	no	0	none	no	skipped	success	success	success	skipped	success	single	initial
1FOR	P00742	16-50	yes	yes	no	0	none	no	skipped	success	success	success	skipped	success	single	initial
1I00	P04818	6-306	no	yes	no	0	none	no	skipped	success	success	success	skipped	success	single	initial
1UOU	P19971	33-480	no	yes	no	2	2|8	yes	success	success	success	success	skipped	success	best_complete	modeller
1W4R	P04183	18-191	yes	yes	no	0	none	no	skipped	success	success	success	skipped	success	single	initial
2AFX	Q16769	33-361	yes	yes	no	2	6|6	yes	failed	success	success	success	skipped	success	gaps	gaps
2PIT	P19793	229-458	no	yes	no	1	19	yes	success	success	success	success	skipped	success	gaps.large_gap_complete	gaps | alphafold
2W8Y	P06401	683-932	no	yes	no	0	none	no	skipped	success	success	success	skipped	success	single	initial
2WCG	P04062	0-497	no	yes	no	1	3	yes	success	success	success	success	skipped	success	gaps.best_complete	gaps | alphafold
3BC3	P07711	1-220	no	yes	yes	2	5|5	yes	failed	success	success	success	skipped	success	gaps	gaps
3EWJ	P78536	220-474	yes	yes	no	0	none	no	skipped	success	success	success	skipped	success	single	initial
3FDN	O14965	126-394	no	yes	no	1	8	yes	success	success	success	success	skipped	success	gaps.best_complete	gaps | modeller
3FRG	Q07343	153-487	yes	yes	no	0	none	no	skipped	success	success	success	skipped	success	single	initial
3K5E	P52732	18-364	yes	yes	no	2	2|15	yes	success	success	success	success	skipped	success	gaps.large_gap_complete	gaps | alphafold
3L3M	P09874	2-349	no	yes	no	0	none	no	skipped	success	success	success	skipped	success	single	initial
3LBK	Q00987	26-110	no	yes	no	0	none	no	skipped	success	success	success	skipped	success	single	initial
3OLL	Q15788	262-498	no	yes	yes	1	2	yes	success	success	success	success	skipped	success	gaps.best_complete	gaps | alphafold
3R04	P11309	35-305	no	yes	no	0	none	no	skipped	success	success	success	skipped	success	single	initial
3RM2	P00734	16-246	yes	yes	no	1	2	yes	success	success	success	success	skipped	success	best_complete	modeller

Figure 1. Example overview sheet from pipeline.xlsx. The screenshot is included as a practical reading aid rather than decoration.

11. The seven-sheet workbook structure

The workbook is intentionally split into seven sheets because each one answers a different user question. Collapsing them into one sheet would look simpler but would make interpretation slower and harder. The sheet split follows how a human actually reviews a batch.

Sheet	What the user should use it for
overview	First-pass triage and high-level interpretation of each record.
paths	Direct navigation to directories and important files.
components	Check for metals, ligands, and nonstandard residues.
gaps_and_variants	Understand incompleteness, filler outcome, and retained policy.
chemistry	Inspect protonation, AMBER renaming, capping, and termini processing.
prepared_outputs	See final output families and prepared structure destinations.
full_state	Audit the flat state table with non-empty columns only.

12. Status colors and semantic colors

FRUTON separates process status from domain semantics. This is important because not every yellow or red cell means the same kind of issue. Some colors indicate what happened during processing; other colors indicate what kind of chemistry or structural caution the record carries.

Color system	Example	Meaning
Status colors	success, warning, required, skipped, failed	Process-state interpretation.
Green semantic yes/no	has_ligands = no, has_metals = no	Normal or favorable state.
Yellow semantic	has_gaps = yes	Structural caution without automatic failure.
Red semantic	has_metals = yes or has_nonstandard_residues = yes	Cases that need closer human inspection.
Grey semantic	skipped	Deliberate inactivity, not silent omission.

13. Interpreting outputs on disk

The workbook tells the user what happened; the files on disk show the concrete evidence. The user should move between both. If a row in the workbook claims that a complete route exists, the corresponding prepared path and component files should exist on disk. If a status failed, the logs directory should contain the module-specific explanation.

FRUTON is strongest when it is used as an inspectable workspace. The correct habit is therefore not to trust one summary cell in isolation, but to move from workbook row to file path to log when a case matters.

14. How to review one protein record

The fastest reliable review path is always the same: identify the row in the overview sheet, check composition flags, inspect gap and chemistry statuses, read the retained variant fields, and then follow the paths or logs only where needed. This keeps review efficient without throwing away traceability.

1. Open the overview sheet and locate the row by pdb_id.
2. Check composition flags first: metals, ligands, nonstandard residues.
3. Check structural incompleteness: n_gaps, gap_sizes, has_gaps, filler.status.
4. Check chemistry progression: protonation, AMBER renaming, termini, internal capping, prepared structure.
5. Read prepared_structure.variant and available_models.
6. Open the paths sheet if you need the exact directories or output files.
7. Open logs/<PDB_ID>/ when a warning or failure needs explanation.
8. Use full_state only when a detailed audit is required.

15. Logs and troubleshooting

FRUTON writes per-protein, per-module logs under data/proteins/logs. These logs are not just developer artifacts. They are user-facing debugging aids. When a record fails, the first move should be to inspect the relevant module log for that PDB ID instead of rerunning blindly.

A useful troubleshooting pattern is to start from the workbook, identify the stage that failed or looks suspicious, then open the matching module log. Common places to inspect are filler.log, protonation.log, amber_renaming.log, termini-related outputs, and any variant-specific logs associated with the failing route.

Symptom	First place to inspect
No UniProt context or missing mappings	fasta_files and sequence_alignment outputs/logs.
Unexpected gap policy	gaps summary fields and filler-related logs.
Chemistry stage failed	protonation, AMBER renaming, termini, and prepared-structure logs.
Workbook looks inconsistent	pipeline.json, paths sheet, and corresponding prepared outputs.

16. Practical operating model

The best way to use FRUTON is iterative. Prepare a small input set, run the pipeline, inspect the workbook and selected logs, adjust the input list or stage assumptions if needed, and then rerun. The framework is designed to preserve enough evidence on disk that iteration remains practical.

For routine usage, the workbook should be treated as the main dashboard and the logs as the explanation layer. For scientific reporting or downstream workflow selection, the prepared outputs and available_models field become the handoff layer.

17. Short identity appendix

In short form: FRUTON is a named protein-preparation framework. The acronym explains the technical scope, the logo gives the project a memorable F-shaped fruit signet with protein-themed interior motifs, and Joseph S. Fruton serves as the scientific namesake. That is enough context for a manual. The operational core remains the workflow, the state model, the workbook, and the inspectable outputs.

18. Final takeaway

A good way to remember FRUTON is this: it is not only a pipeline that produces prepared structures. It is a framework that records why a given structural family was retained, keeps the evidence on disk, and exposes that decision in a workbook a user can actually read.

If the user knows the input table, the run entry point, the variant policy, the workbook logic, and the log locations, they can operate FRUTON effectively. The rest of the project identity is useful background, but not the core of usage.